

1. Total cost

$$T(n) = \sum_{i=1}^n 1 + \sum_{\substack{i \leq n \\ i=2^j}} (i-1) = n + (2^{\lfloor \log_2 n \rfloor + 1} - 1) - (\lfloor \log_2 n \rfloor + 1) < 3n$$

So amortized cost per operation is $T(n)/n = O(1)$ (in fact, < 3).

2.

Pay 1 for the actual stack op

Save 1 as credit

After any k ops you have k credits, which pays for the backup copy (cost $\leq k$)

So n ops costs $\leq 2n = O(n)$

3. Potential function (with n = current heap size):

$$\Phi(n) = \begin{cases} 0, & n = 0 \\ c n \log n, & n \geq 1 \end{cases}$$

- INSERT: real = $O(\log n)$, and $\Delta\Phi = O(\log n) \Rightarrow$ amortized $O(\log n)$.
- EXTRACT-MIN: real = $O(\log n)$, and $\Delta\Phi = -\Omega(\log n)$. Choose c large enough so the drop in potential cancels the real $O(\log n)$ work $\Rightarrow$ amortized $O(1)$

4. Amortized analysis spreads the occasional expensive “restructure” over many cheap operations: we charge each operation slightly more than its usual cost and store the extra as credit. When a restructure happens, we pay for it using the accumulated credit, so the average cost per operation stays low over the whole sequence.

- 5 Aggregate: bit j flips at most $n/2^j$ times, so total flips

$$\leq n + \frac{n}{2} + \frac{n}{4} + \dots < 2n$$

Amortized $< 2 = O(1)$.

- Accounting: charge 2 per increment; pay 1 for the required 0→1 flip, store 1 credit on that bit to pay for its future 1→0 flip ⇒ constant amortized.
- Potential: $\Phi = \# \text{ of 1s}$. If t trailing 1s flip to 0, real cost = $t + 1$, and $\Delta\Phi = 1 - t$.
Amortized

$$(t + 1) + (1 - t) = 2.$$

Splay trees + Zag-Zig

Use rank $r(v) = \log(\text{size}(v))$ and potential $\Phi = \sum_v r(v)$.

In a **zig-zag** step (two rotations), the amortized cost satisfies the standard bound:

$$\hat{c} \leq 3(r'(x) - r(x)) + 1,$$

where $r'(x)$ is x 's rank after the step.

Summing over all splay steps in one operation telescopes:

$$\sum \hat{c} \leq 3(r(\text{root}) - r(x)) + O(1) \leq 3\log n + O(1),$$

so each splay (and thus each operation) is $O(\log n)$ amortized even if some single operations are slower.